Departamento de
Tecnología Digital

Tecsup TECNOLOGÍA CON SENTIDO



LABORATORIO N° 01

Diseño y Desarrollo de Simuladores y Videojuegos

DOCENTE:

Bestard Aroche, Yunior

CURSO:

Desarrollo de Aplicaciones Empresariales
4 - C24 - Sección CD

DESARROLLO DE APLICACIONES EMPRESARIALES:

Capacidades

Capacidad Terminal 1

- Identificar las características de Django como framework de desarrollo web.

Capacidad Terminal 2

- Instalar y configurar el entorno de desarrollo (Python, entorno virtual y Django).

Capacidad Terminal 3

- Crear una aplicación Web en Django, reconociendo su estructura y elementos principales (proyecto, app, modelos, vistas, URLs y administrador).

I. Seguridad

- En este laboratorio está prohibida la manipulación del hardware, conexiones eléctricas o de red. Así como la ingesta de alimentos y bebidas.
- Ubicar maletines y/o mochilas en lugar destinado para tal fin.
- Dejar la mesa de trabajo y la silla utilizada limpias y ordenadas.

II. Fundamento teórico

- Revise el material de la semana correspondiente antes del desarrollo del laboratorio.

III. Normas empleadas

- *No aplica*

IV. Recursos

- En este laboratorio, cada estudiante trabajará con una computadora con Windows 11.
- Los softwares requeridos ya se encuentran instalados en los laboratorios.
- Se requiere Python 3.10 o superior, Django 5, un editor de código (Visual Studio Code) y una cuenta de GitHub para el control de versiones.

V. Metodología para el desarrollo de la tarea

- El desarrollo del laboratorio es individual

VI. Procedimiento

Introducción al laboratorio

En este laboratorio construirás la primera versión de tu “Sistema Empresarial”: una aplicación Web en Django que gestiona un catálogo simple de ítems (por ejemplo, productos). Este proyecto servirá de base para las siguientes sesiones del curso.

Ejercicios

Desarrolla, en orden, los siguientes ejercicios. No copies comandos sin comprenderlos: cada paso debe quedar evidenciado con una captura de pantalla y una breve explicación.

1. Preparar el entorno de trabajo. Crea la carpeta del proyecto (django_project), crea y activa un entorno virtual de Python (venv), y dentro de él crea la carpeta src/ que contendrá el código fuente.

2. Instalar Django. Dentro del entorno virtual activado, instala Django (versión 5) usando pip y verifica la instalación consultando su versión.

3. Crear el proyecto con configuración separada. Usando django-admin, crea un proyecto llamado config dentro de la carpeta src/, de forma que manage.py quede en src/ y la configuración (settings.py, urls.py, etc.) quede en src/config/.

4. Crear y registrar la aplicación core. Crea la aplicación core con manage.py startapp y regístrala en la lista INSTALLED_APPS de src/config/settings.py.

5. Definir el modelo Item. En src/core/models.py define un modelo Item con los campos name (texto), description (texto largo, opcional) y created_at (fecha/hora de creación automática). Genera y aplica las migraciones correspondientes.

6. Crear la vista y las URLs. Implementa en src/core/views.py una vista item_list que obtenga todos los Item y los envíe a una plantilla. Configura las URLs de la app (core/urls.py) y enlázalas en src/config/urls.py mediante include().

7. Crear las plantillas. Crea una plantilla base (base.html) con una estructura HTML general, y una plantilla core/item_list.html que herede de la base y muestre el listado de ítems (usa {% for %} y {% empty %}).

8. Configurar el administrador y cargar datos. Registra el modelo Item en src/core/admin.py, crea un superusuario, ingresa al panel de administración y registra al menos dos ítems de prueba.

9. Verificar el funcionamiento. Levanta el servidor de desarrollo (runserver) y comprueba que la página principal (http://127.0.0.1:8000/) muestra el listado de ítems, y que el panel de administración (/admin/) funciona correctamente.

10. Documentar y subir el proyecto. Genera el archivo requirements.txt con las dependencias del proyecto, crea un README.md que explique la estructura y el modo de instalación/ejecución, y sube el proyecto a un repositorio de GitHub.

Reto (opcional)

- Agrega estilos CSS a la plantilla base para mejorar la presentación del listado de ítems.
- Agrega un elemento de interactividad en el frontend (por ejemplo, un filtro o buscador con JavaScript).
- Con apoyo de IA, crea uno o más endpoints (API) para el modelo Item y consúmelos desde un frontend construido con HTML, CSS y JavaScript vanilla, o con React.

Entregables

- Informe con esta misma plantilla del laboratorio: capturas de pantalla de cada ejercicio, con una breve explicación de lo realizado.
- Repositorio: enlace al repositorio de GitHub con el proyecto completo.
- Sustentación: el estudiante debe estar en capacidad de explicar y ejecutar su proyecto en clase.
- Subir al canvas en formato pdf

Solución de los ejercicios

A continuación se presenta una posible solución de referencia para el docente, con los comandos y el código esperado en cada ejercicio.

1-2. Preparar el entorno e instalar Django

```
mkdir django_project && cd django_project
python3 -m venv venv
source venv/bin/activate      # En Windows: venv\Scripts\activate
mkdir src && cd src
pip3 install django
python3 -m django --version
```

3. Crear el proyecto config

```
# Estando dentro de src/
django-admin startproject config .
```

Esto genera manage.py en src/ y la carpeta config/ con settings.py, urls.py, asgi.py y wsgi.py.

4. Crear y registrar la app core

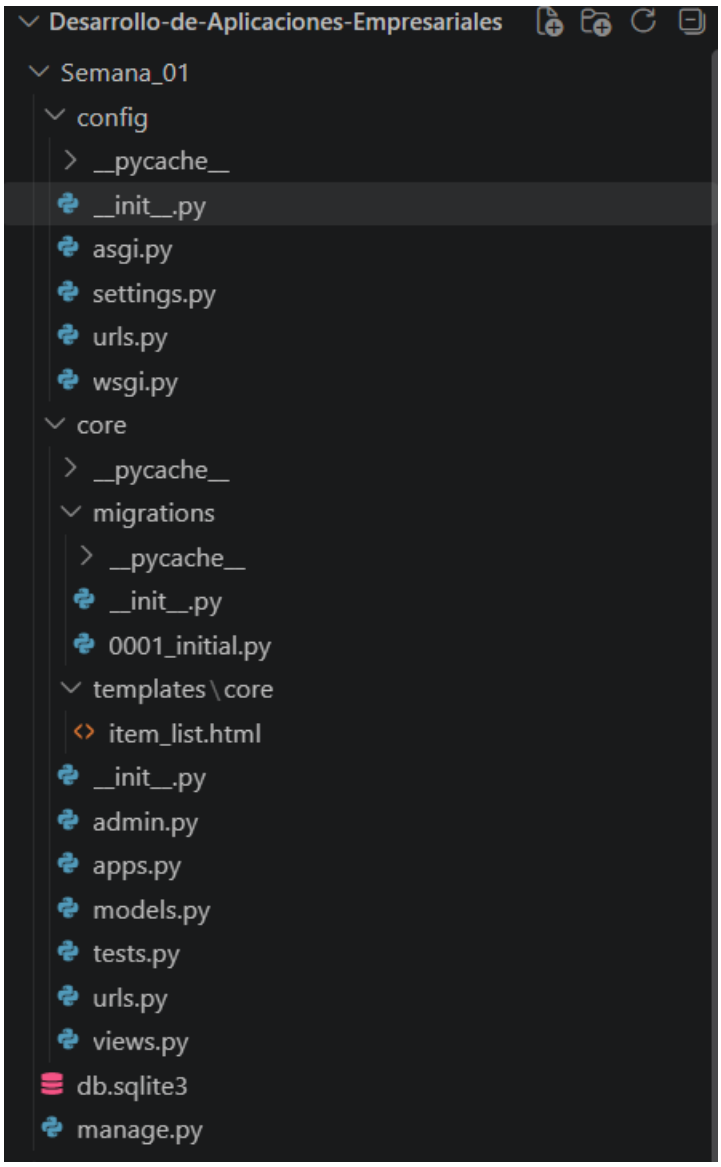
```
python3 manage.py startapp core
```

En src/config/settings.py:

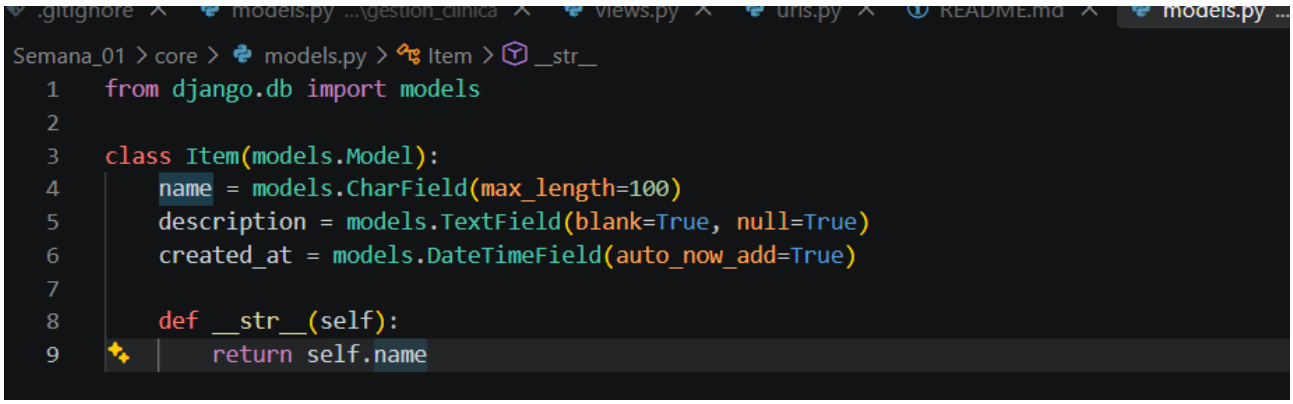
```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'core',  # Nuestra aplicación
]
```

EVIDENCIA:

ESTRUCTURA:



CODIGO:



Semana_01 > core > urls.py > ...

```python
from django.urls import path
from . import views

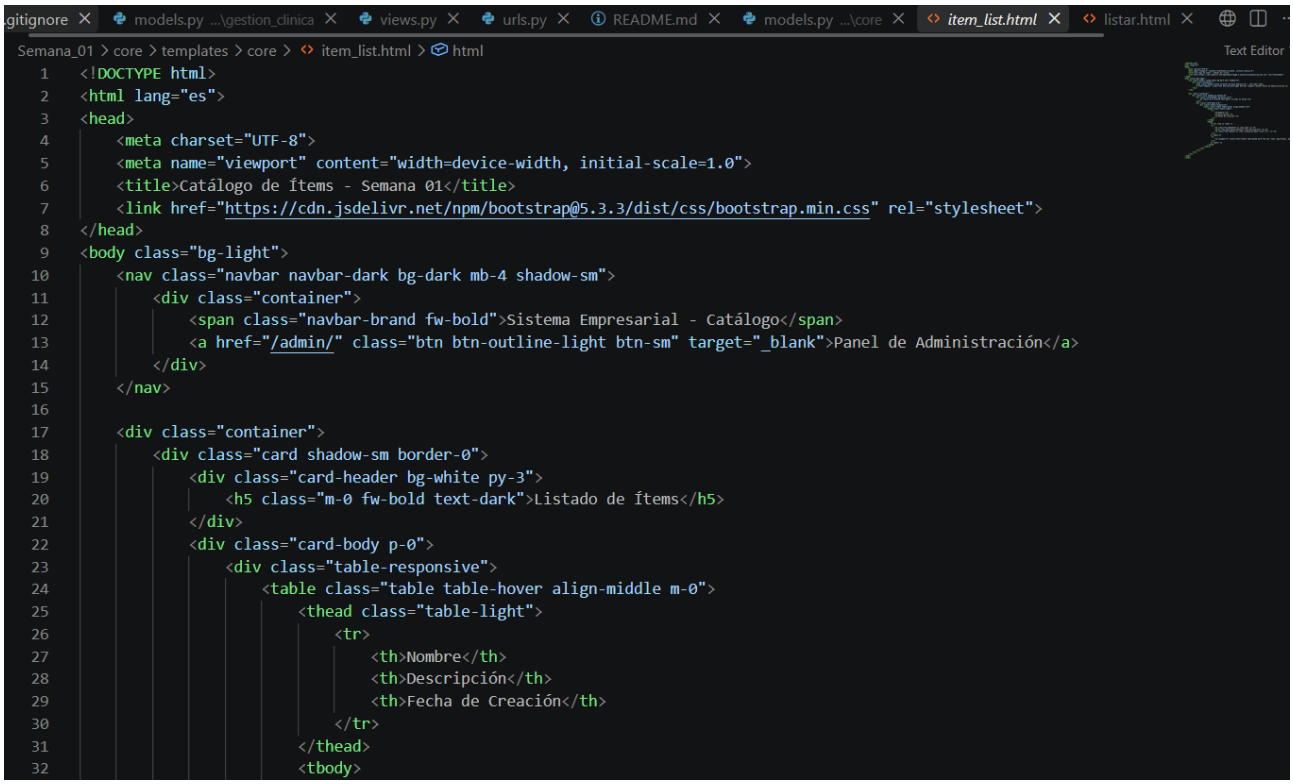
urlpatterns = [
    path('', views.item_list, name='item_list'),
]
```

Semana_01 > core > views.py > item_list

```python
from django.shortcuts import render
from .models import Item

def item_list(request):
    items = Item.objects.all()
    return render(request, 'core/item_list.html', {'items': items})
```

HTML



```html
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Catálogo de Ítems - Semana 01</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body class="bg-light">
    <nav class="navbar navbar-dark bg-dark mb-4 shadow-sm">
        <div class="container">
            <span class="navbar-brand fw-bold">Sistema Empresarial - Catálogo</span>
            <a href="/admin/" class="btn btn-outline-light btn-sm" target="_blank">Panel de Administración</a>
        </div>
    </nav>

    <div class="container">
        <div class="card shadow-sm border-0">
            <div class="card-header bg-white py-3">
                <h5 class="m-0 fw-bold text-dark">Listado de Ítems</h5>
            </div>
            <div class="card-body p-0">
                <div class="table-responsive">
                    <table class="table table-hover align-middle m-0">
                        <thead class="table-light">
                            <tr>
                                <th>Nombre</th>
                                <th>Descripción</th>
                                <th>Fecha de Creación</th>
                            </tr>
                        </thead>
                        <tbody>
```

```
        <div class="card shadow-sm border-0">
            <div class="card-body p-0">
                        <tbody>
                            {% for item in items %}
                            <tr>
                                <td class="fw-semibold">{{ item.name }}</td>
                                <td>{{ item.description|default:"Sin descripción" }}</td>
                                <td class="text-muted">{{ item.created_at|date:"d/m/Y H:i" }}</td>
                            </tr>
                            {% empty %}
                            <tr>
                                <td colspan="3" class="text-center text-muted py-3">No hay ítems registrados. Agrégalos d
                            </tr>
                            {% endfor %}
                        </tbody>
                    </table>
                </div>
            </div>
        </div>
    </div>
</body>
</html>
```

Conclusión — Laboratorio de la Semana 01

- **Estructura y arquitectura modular: En este primer laboratorio logré sentar las bases del desarrollo backend en Django configurando una arquitectura desacoplada. Comprendí la importancia de separar las configuraciones globales del proyecto (dentro de la carpeta config) de la lógica de negocio de la aplicación (core). Al principio el objetivo era preparar un entorno profesional y aislado sin mezclar responsabilidades en un solo archivo.**
- **Persistencia y panel de administración: Pude comprobar cómo el ORM de Django simplifica el trabajo con bases de datos al crear el modelo Item en models.py. En lugar de redactar sentencias SQL manuales, utilicé makemigrations y migrate para que el framework construyera las tablas físicas automáticamente en SQLite. Además, habilitar el modelo dentro de admin.py y crear un superusuario me permitió gestionar los primeros registros de forma rápida y segura desde el panel nativo (/admin/), sin tener que armar formularios desde el inicio.**
- **Flujo y visualización de datos: Implementé el recorrido completo conectando las rutas mediante urls.py (usando include()) hacia una vista en views.py que consultó los datos con Item.objects.all(). Finalmente, rendericé esa información en una plantilla HTML con Bootstrap usando el motor DTL ({% for %} y {% empty %}), logrando un catálogo web funcional y dinámico.**